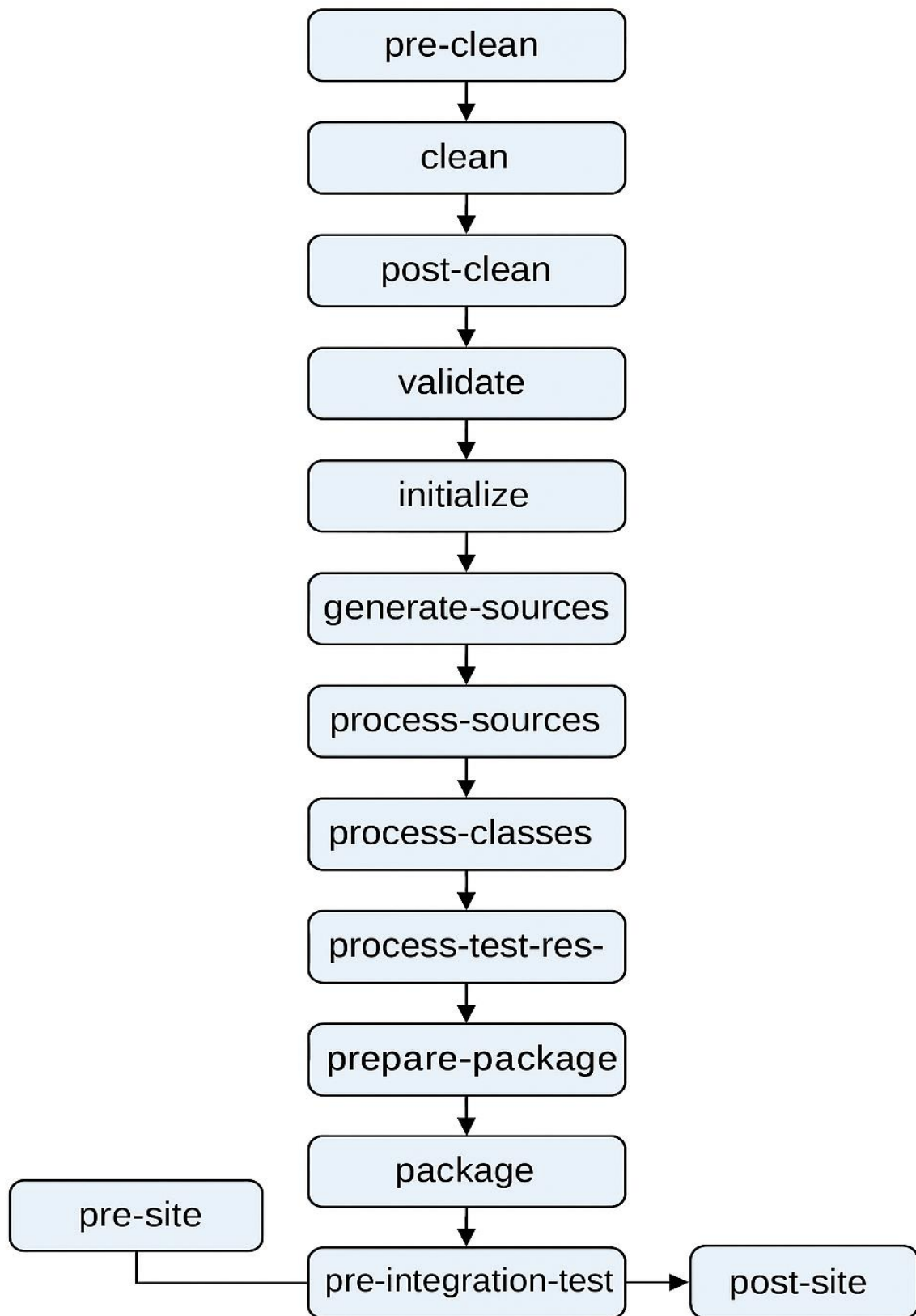




Great question — let's break it into **two parts**:

1. **Understanding pom.xml**
2. **Complete list of Maven commands with examples in a table (with Windows 11 and Ubuntu 22.04 run commands).**

1. What is pom.xml in Maven?

The **Project Object Model (pom.xml)** is the fundamental unit of a Maven project. It defines the **project configuration, dependencies, plugins, and build settings**.

Key Sections of pom.xml

Section	Purpose	Example
<modelVersion>	POM model version	<code><modelVersion>4.0.0</modelVersion></code>
<groupId>	Unique ID for project group/org	<code><groupId>com.myapp</groupId></code>
<artifactId>	Name of the project/module	<code><artifactId>demo-app</artifactId></code>
<version>	Version of the artifact	<code><version>1.0.0</version></code>
<packaging>	Defines output type (jar, war, pom)	<code><packaging>jar</packaging></code>
<dependencies>	External libraries needed	<code><dependency> <groupId>junit</groupId> <artifactId>junit</artifactId> <version>4.13</version> <scope>test</scope></dependency></code>
<build>	Build configurations (plugins, goals)	<code><build><plugins> ... </plugins></build></code>

So, pom.xml acts like a **blueprint** that tells Maven how to build, package, and manage dependencies of your project.

2. Maven Commands Table

Below is the **complete list of commonly used Maven commands**, with description, syntax, and how to run on **Windows 11** vs **Ubuntu 22.04**.

Command	Purpose	Example	Windows 11	Ubuntu 22.04
mvn -v	Check Maven version	mvn -v	mvn -v in Command Prompt / PowerShell	mvn -v in Terminal
mvn clean	Deletes target/ directory (clean build)	mvn clean	mvn clean	mvn clean
mvn compile	Compiles source code in src/main/java	mvn compile	mvn compile	mvn compile
mvn test	Runs unit tests	mvn test	mvn test	mvn test
mvn package	Packages compiled code into .jar or .war	mvn package	mvn package	mvn package
mvn install	Installs package into local repository (~/.m2/repository)	mvn install	mvn install	mvn install
mvn deploy	Deploys to remote repo (Nexus/Artifactory)	mvn deploy	mvn deploy	mvn deploy
mvn site	Generates documentation for project	mvn site	mvn site	mvn site
mvn dependency:tree	Shows dependency hierarchy	mvn dependency:tree	mvn dependency:tree	mvn dependency:tree

mvn validate	Validates pom.xml before build	mvn validate	mvn validate	mvn validate
mvn verify	Runs integration tests & checks project	mvn verify	mvn verify	mvn verify
mvn clean install	Clean + compile + test + package + install	mvn clean install	mvn clean install	mvn clean install
mvn clean package -DskipTests	Skips running tests	mvn clean package -DskipTests	mvn clean package -DskipTests	mvn clean package -DskipTests
mvn archetype:generate	Create new Maven project from template	mvn archetype:generate -DgroupId=com.example -DartifactId=myapp -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false	Same	Same
mvn exec:java -Dexec.mainClass="com.myapp.Main"	Run a Java class directly	mvn exec:java -Dexec.mainClass="com.myapp.Main"	Same	Same

How to Run on Local Systems

Windows 11

1. Install **Java JDK** (check with `java -version`).
2. Install **Maven** → add MAVEN_HOME and update PATH.
3. Open **Command Prompt** / **PowerShell** → run commands (mvn clean install).

Ubuntu 22.04

```
sudo apt update
sudo apt install maven -y
```

```
mvn -v # verify installation
```

Run commands in **Terminal** (mvn clean install).

Would you like me to also prepare a **step-by-step real-world demo** with a **sample pom.xml + running Maven commands on Windows and Ubuntu** (so you can practice hands-on)?

Perfect ☑ Let's do a **real-world demo** of using Maven with a **step-by-step example**. We'll build a simple **Java Hello World project** with Maven, using a sample pom.xml, and I'll show how to run commands on **Windows 11** and **Ubuntu 22.04**.

Step 1: Prerequisites

Install Java JDK

- Windows: Download from [Oracle](#) or use OpenJDK.
- Ubuntu:
 - `sudo apt update`
 - `sudo apt install openjdk-17-jdk -y`
 - `java -version`

Install Maven

- Windows: Download [Maven](#), unzip, and set MAVEN_HOME + add to PATH.
 - Ubuntu:
 - `sudo apt install maven -y`
 - `mvn -v`
-

Step 2: Create a New Maven Project

Windows 11 (PowerShell / CMD)

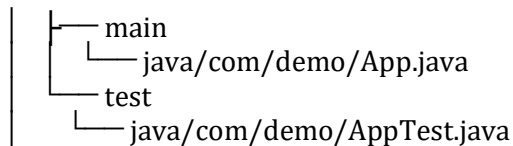
```
mvn archetype:generate -DgroupId=com.demo -DartifactId=hello-maven -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```

Ubuntu 22.04 (Terminal)

```
mvn archetype:generate -DgroupId=com.demo -DartifactId=hello-maven -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```

This creates a folder hello-maven with structure:

```
hello-maven/  
├── pom.xml  
└── src
```



```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/maven-v4_0_0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>com.demo</groupId>

  <artifactId>hello-maven</artifactId>

  <version>1.0-SNAPSHOT</version>

  <packaging>jar</packaging>

  <name>hello-maven</name>

  <url>http://maven.apache.org</url>

  <dependencies>

    <!-- Example: JUnit for testing -->

    <dependency>

      <groupId>junit</groupId>

      <artifactId>junit</artifactId>

      <version>4.13.2</version>

      <scope>test</scope>

    </dependency>

  </dependencies>
```

```
<build>
  <plugins>
    <!-- Maven Compiler Plugin -->
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.11.0</version>
      <configuration>
        <source>17</source>
        <target>17</target>
      </configuration>
    </plugin>

    <!-- Maven Exec Plugin (to run Java class) -->
    <plugin>
      <groupId>org.codehaus.mojo</groupId>
      <artifactId>exec-maven-plugin</artifactId>
      <version>3.1.0</version>
      <configuration>
        <mainClass>com.demo.App</mainClass>
      </configuration>
    </plugin>
  </plugins>
</build>
```

</project>

```
package com.demo;

public class App {

    public static void main(String[] args) {

        System.out.println("Hello, Maven World!");

    }

}
```

Step 5: Running Maven Commands

Task	Windows 11 (CMD/PowerShell)	Ubuntu 22.04 (Terminal)
Clean old build	mvn clean	mvn clean
Compile code	mvn compile	mvn compile
Run tests	mvn test	mvn test
Package into JAR	mvn package	mvn package
Install to local repo	mvn install	mvn install
Run application	mvn exec:java - Dexec.mainClass="com.demo.App"	mvn exec:java - Dexec.mainClass="com.demo.App"

After packaging, the JAR will be in:

target/hello-maven-1.0-SNAPSHOT.jar

Run directly with Java:

```
java -cp target/hello-maven-1.0-SNAPSHOT.jar com.demo.App
```

Step 6: Expected Output

When running the app:

Hello, Maven World!

Now you have a **working real-world Maven project**

- Built with pom.xml
 - Runs on **Windows 11 and Ubuntu 22.04**
 - Demonstrates **all Maven phases**: clean → compile → test → package → install → run
-

Do you want me to also make a **table of Maven lifecycle phases (default, clean, site)** with what exactly happens in each step (so you can connect commands to the lifecycle)?

Perfect! let's now break down **Maven lifecycle phases** in a **table format**.

Maven has **3 built-in lifecycles**:

1. **default** (build lifecycle) → compile, test, package, install, deploy
 2. **clean** (clean lifecycle) → cleaning up previous builds
 3. **site** (site lifecycle) → generating documentation
-

Maven Lifecycle Phases Table

Lifecycle	Phase	Description	Typical Command
Clean	pre-clean	Tasks before cleaning	mvn pre-clean
	clean	Deletes target/ directory (old builds)	mvn clean
	post-clean	Tasks after cleaning	mvn post-clean
Default (Build)	validate	Validate pom.xml and project structure	mvn validate
	initialize	Initialize build (set properties, create dirs)	mvn initialize
	generate-sources	Generate source code (from annotations, codegen)	mvn generate-sources
	process-sources	Process source code (filtering, formatting)	mvn process-sources

	generate-resources	Generate resources (e.g., config files)	mvn generate-resources
	process-resources	Copy resources to target/classes	mvn process-resources
	compile	Compile source code (src/main/java)	mvn compile
	process-classes	Post-processing of compiled classes (bytecode enhancement)	mvn process-classes
	generate-test-sources	Generate test sources	mvn generate-test-sources
	process-test-sources	Process test sources (filter, preprocess)	mvn process-test-sources
	generate-test-resources	Generate test resources	mvn generate-test-resources
	process-test-resources	Copy test resources to target/test-classes	mvn process-test-resources
	test-compile	Compile test source code (src/test/java)	mvn test-compile
	process-test-classes	Post-process compiled test classes	mvn process-test-classes
	test	Run unit tests (JUnit/TestNG)	mvn test
	prepare-package	Do tasks before packaging (e.g., code gen)	mvn prepare-package
	package	Package into .jar or .war inside target/	mvn package
	pre-integration-test	Setup integration test environment	mvn pre-integration-test
	integration-test	Run integration tests	mvn integration-test

	post-integration-test	Cleanup after integration tests	mvn post-integration-test
	verify	Run checks to verify package is valid	mvn verify
	install	Install .jar/.war into local repo (~/.m2/)	mvn install
	deploy	Deploy artifact to remote repo (Nexus/Artifactory)	mvn deploy
Site	pre-site	Tasks before site generation	mvn pre-site
	site	Generate project documentation (HTML reports)	mvn site
	post-site	Tasks after site generation	mvn post-site
	site-deploy	Deploy site to web server	mvn site-deploy

Important Notes

- Running **mvn compile** will also trigger all **previous phases** (validate → initialize → generate-sources → process-sources → compile).
 - Running **mvn install** runs the **entire default lifecycle** up to install.
 - **Phases vs Goals:**
 - *Phase* = step in lifecycle (compile, test, package).
 - *Goal* = specific task bound to phase (e.g., compiler:compile, surefire:test).
-

Do you also want me to make a **visual diagram (flow chart)** of these lifecycles, so you can quickly understand the flow for interviews?